CUDA Online Softmax 笔记

> 参考资料：AIInfraGuide · 5.2 CUDA Online Softmax 实现 代码：本目录 `Operator/OSoftmax/`（V0 → V5，每个版本一个 `.cu`，文件头注释保留了当次实测数据） 实测环境：`M = N = 4096`，`block = 256`，`warmup = 5`，`iterations = 200`，GPU 为 **RTX 2080 Ti（sm_75，峰值带宽 ~616 GB/s）** 附加文件：`ThreePass.cu`（三遍 Safe Softmax 基线，用于和第 9 条的内存吞吐对比）

一、先把「几遍扫描」这件事说清楚

Softmax 要算的是 $\mathrm{softmax}(x)_i = \dfrac{e^{x_i - m}}{\sum_j e^{x_j - m}}$，其中 $m = \max_j x_j$。减 m 是纯数值稳定措施：它不改变结果，但保证指数不溢出。

关键点在于：**m 和归一化分母 $d = \sum_j e^{x_j - m}$ 都依赖整行的统计量，而 y_i 又需要逐元素算，所以「求统计量」和「写输出」之间天然存在一次数据依赖**。不同实现方案的差别，就是「为了拿到 m 和 d，输入 x 要读几遍」。

方案	扫描结构	读 x	写 y	每元素 DRAM 流量	M=N=4096 总流量
三遍 Safe Softmax	max → sum → normalize	3 遍	1 遍	16 Byte	268.4 MB
两遍 Online Softmax（V0/V1/V2/V4）	(max,sum) 一次融合 → normalize	2 遍	1 遍	12 Byte	201.3 MB
一遍寄存器缓存（V3/V5）	缓存整行 → 一次融合求 (m,d) → 直接算 y	1 遍	1 遍	8 Byte	134.2 MB

> 说明：前两列是按「不破坏依赖关系」的等价代码结构划分的。V3/V5 的第二次遍历读的是寄存器，而不是 global memory，所以真实 DRAM 流量退化成 1 读 1 写——这也是「One-Pass」这个名字的由来。

这就是 Online Softmax 的全部价值所在：**它不改变数学结果，只是把「求 max」和「求 sum」这两遍合并成一遍**，把一个 memory-bound 算子的访存量从 4 段降到 3 段（寄存器缓存版降到 2 段）。

二、逐条对照自我检验清单

1. Online Softmax 相比三遍 Safe Softmax，省了哪一遍扫描，原理是什么

三遍版本的三次遍历各自独立：

```
float m = -INFINITY;
for (int i = 0; i < N; ++i) m = fmaxf(m, x[i]);        // 第 1 遍：求 max
float d = 0.0f;
for (int i = 0; i < N; ++i) d += expf(x[i] - m);        // 第 2 遍：求 sum
float inv = 1.0f / d;
for (int i = 0; i < N; ++i) y[i] = expf(x[i]-m)*inv;    // 第 3 遍：归一化
```

三遍版的死结是：**第 2 遍的求和必须等第 1 遍的 max 出来才能开始**，因为 e^{x_i-m} 里的 m 还不知道，而一旦拿了错的 m，后面所有项都要重算。

Online Softmax 看穿了这一点：m 不需要一次到位。**如果我们允许 d 是一个「相对于当前临时 max」的部分和，就能一边扫描一边修正它**。于是第 1、2 遍被融合成一趟：

```
// 对应 V0 的第 1 遍
float m = -INFINITY, d = 0.0f;
for (int i = 0; i < N; ++i) {
    float m_new = fmaxf(m, x[i]);
    d = d * expf(m - m_new) + expf(x[i] - m_new);   // 见第 2 条递推
    m = m_new;
}
```

省下的就是原来那趟「为了求 m 而单独读一遍 x」的扫描。**代价是每次 max 更新时都要给历史累加项乘一个 $e^{m_{\text{old}}-m_{\text{new}}}$ 的缩放因子**，纯粹是寄存器级别的浮点运算，不产生额外访存。

在 `M=N=4096` 下，省一趟读 = 少搬 67.1 MB，$268.4 \to 201.3$ MB，**访存量下降 25%**。

2. 手动推导 $d_{k+1} = d_k \cdot e^{m_k - m_{k+1}} + e^{x_{k+1}-m_{k+1}}$

先写清楚定义。处理到第 k 个元素时：

- 前缀最大值 $m_k = \max(x_1, \dots, x_k)$
- 前缀和 $d_k = \sum_{i=1}^{k} e^{x_i - m_k}$

要证的目标是

$$d_{k+1} = d_k \cdot e^{m_k - m_{k+1}} + e^{x_{k+1}-m_{k+1}}$$

推导：按定义展开 d_{k+1}，把 $i \le k$ 的部分和最后一项 $i = k+1$ 拆开：

$$d_{k+1} = \sum_{i=1}^{k} e^{x_i - m_{k+1}} + e^{x_{k+1}-m_{k+1}}$$

对前 k 项做一次指数恒等变形，凑出 $x_i - m_k$：

$$e^{x_i - m_{k+1}} = e^{x_i - m_k + m_k - m_{k+1}} = e^{x_i - m_k} \cdot e^{m_k - m_{k+1}}$$

把公共因子 $e^{m_k - m_{k+1}}$ 提到求和号外：

$$\sum_{i=1}^{k} e^{x_i - m_{k+1}} = e^{m_k - m_{k+1}} \sum_{i=1}^{k} e^{x_i - m_k} = d_k \cdot e^{m_k - m_{k+1}}$$

代回即得目标式。□

为什么这个式子数值安全：$m_{k+1} = \max(m_k, x_{k+1}) \ge m_k$ 且 $\ge x_{k+1}$，所以两个指数都 ≤ 0，两个 $e^{\cdot} \in (0, 1]$，不会上溢；两个因子中必有一个等于 1（谁成为新 max 谁对应的指数是 $e^0 = 1$，另一个是被缩放项）。这正好对应代码里的两种情形：

- $x_{k+1} \le m_k$：$m_{k+1} = m_k$，$e^{m_k - m_{k+1}} = 1$，退化成 $d \mathrel{+}= e^{x_{k+1}-m_k}$；
- $x_{k+1} > m_k$：$m_{k+1} = x_{k+1}$，$e^{x_{k+1}-m_{k+1}} = 1$，历史项整体被压缩为 $d \cdot e^{m_k - x_{k+1}}$。

初始条件：$m_0 = -\infty$ 时 $d_0 = 0$，第一次迭代 $d_1 = 0 \cdot e^{-\infty - x_1} + e^{x_1 - x_1} = 1$，自洽。注意 $0 \cdot e^{-\infty}$ 里 $-\infty - x_1 = -\infty$、$e^{-\infty} = 0$，这是合法的，不产生 NaN。（但**两个中性元相遇时会出问题**，见第 4 条。）

3. 两个局部 (m_1, d_1)、(m_2, d_2) 合并成全局 (m, d) 的代码

设局部 1 覆盖集合 S_1、局部 2 覆盖 S_2，则

$$m = \max(m_1, m_2), \qquad d = d_1 \cdot e^{m_1 - m} + d_2 \cdot e^{m_2 - m}$$

推导逻辑和第 2 条完全一样，只是把「新增一个元素」换成了「并入一整个块」。

```
// 与 V1~V5 中 warpReduceOnline 的核心语句一致
__device__ __forceinline__ void merge_md(float &m, float &d, float m2, float d2) {
    float m_new = fmaxf(m, m2);
    // 两项都相对新 max 做缩放；谁大谁对应的 expf 等于 1
    d = d * expf(m - m_new) + d2 * expf(m2 - m_new);
    m = m_new;
}
```

调用完毕后 `m` 是两段的全局 max，`d` 是**以该 max 为基准**的全局和，可以直接送去做归一化。

以 V1 的 warp 归约为例，每个 lane 先各自算好本 lane 的 (m, d)，再两两合并，5 步后 lane 0 得到整个 warp 的结果：

```
__device__ void warpReduceOnline(float &m, float &d){
    for (int offset = 16; offset; offset >>= 1) {
        float m2 = __shfl_down_sync(0xffffffff, m, offset);
        float d2 = __shfl_down_sync(0xffffffff, d, offset);
        float m_merge = fmaxf(m2, m);
        d = d * expf(m - m_merge) + d2 * expf(m2 - m_merge);
        m = m_merge;
    }
}
```

4. 为什么 (m, d) 的合并满足结合律，因此可以并行

把一次「合并」写成二元算子 $\oplus$：

$$(m_1, d_1) \oplus (m_2, d_2) = \Big(\max(m_1, m_2),\ d_1 e^{m_1 - m^*} + d_2 e^{m_2 - m^*} \Big), \quad m^* = \max(m_1, m_2)$$

证明思路（而不是硬展开三重括号）：注意 $\oplus$ 的结果只取决于两段所覆盖的**元素集合**。若集合 S_1 对应 $(m_1, d_1) = \left(\max_{S_1} x,\ \sum_{i \in S_1} e^{x_i - \max_{S_1} x} \right)$，$S_2$ 对应 (m_2, d_2)，那么

$$(m_1, d_1) \oplus (m_2, d_2) = \left(\max_{S_1 \cup S_2} x,\ \sum_{i \in S_1 \cup S_2} e^{x_i - \max_{S_1 \cup S_2} x} \right)$$

右端只和并集有关，与「先合并谁、怎么分组」无关。而集合并满足结合律与交换律，因此 $\oplus$ 也满足结合律与交换律：

$$((a \oplus b) \oplus c) = (a \oplus (b \oplus c)) = a \oplus b \oplus c$$

（在上面证明里用到了 $m^* = \max(m_1, m_2)$ 时 $e^{m_1 - m^*} \cdot e^{x_i - m_1} = e^{x_i - m^*}$ 这一指数恒等式，这就是缩放因子能「对齐」两段基准的原因。）

结合律 + 交换律 = 可以任意重排、任意分组，这正是 `__shfl_down_sync` 树形归约、shared memory 二级归约、乃至跨 block/跨 kernel 分段归约能在数学上成立的前提。你代码里「warp 内 shuffle → 写 shared memory → warp 0

再 shuffle」两级结构，本质上就是把这个结合算子按树形顺序求值。

中性元与 NaN 陷阱（V1~V5 注释里专门记了这一条）：单位元是 $(-\infty, 0)$。但当**两个单位元相遇**时，

$$\underbrace{-\infty}_{m_1} - \underbrace{(-\infty)}_{m^*} = \mathrm{NaN}$$

NaN 会顺着 shuffle 树的乘法一路污染到 lane 0。所以填充项必须用**有限的极小值**而不是 `-INFINITY`：

```
local_m = (lane_id < num_warp) ? warp_m[lane_id] : -1e30f;
local_d = (lane_id < num_warp) ? warp_d[lane_id] : 0.0f;
```

此时 $e^{-1e30-(\text{有限值})} = 0$，乘上 $d = 0$ 得到 0，是干净的单位元。

5. 用 `__shfl_down_sync` 实现 `warpReduceOnline`

```
__device__ void warpReduceOnline(float &m, float &d){
    for (int offset = 16; offset; offset >>= 1) {          // 16,8,4,2,1 共 5 步 = log2(32)
        float m2 = __shfl_down_sync(0xffffffff, m, offset); // 取本 lane + offset 的 m
        float d2 = __shfl_down_sync(0xffffffff, d, offset); // 取本 lane + offset 的 d
        float m_merge = fmaxf(m2, m);
        d = d * expf(m - m_merge) + d2 * expf(m2 - m_merge);
        m = m_merge;
    }
}
```

要点：

- **两条数据必须一起 shuffle**。(m, d) 是一个整体，只搬 `m` 不搬 `d` 会得到基准不一致的和，结果全错。这也是「合并算子」概念的意义——它必须作为一个不可拆的二元组来传递。
- **每次归约步长减半**是标准的树形规约：offset=16 时 lane 0..15 合并自己的下半区，offset=8 时再合并，共 $\log_2 32 = 5$ 步后 lane 0 持有全 warp 32 个 lane 的合并结果。其他 lane 的结果是「以它自己为根的子段」的合并值，没用但也不出错。
- `0xffffffff` 是掩码，表示 warp 32 个 lane 全部参与。**warp 内必须无分支发散**，否则掩码对应的 lane 不在场会导致未定义行为——这就是为什么填充逻辑写成「选 `-1e30f`」而不是 `continue` 跳过。
- shuffle 走的是寄存器到寄存器，**不经过 shared memory，也不产生 bank conflict**，比「写 shared memory → `__syncthreads()` → 再读」快得多。只有跨 warp 时才退化成 shared memory（代码里的第二级）。

执行过程（4 个值的简化示意，32 个值同理）：设各 lane 的 (m, d) 依次为 $(1,1), (3,1), (2,1), (0,1)$。

步	offset	lane0 合并后	lane1	lane2	lane3
初始	—	(1,1)	(3,1)	(2,1)	(0,1)
1	2	merge((1,1),(2,1))=(2,2)	merge((3,1),(0,1))=(3,2)	—	—
2	1	merge((2,2),(3,2))=(3, 2・e^{2-3}+2・e^0)=(3, 2/e+2)	—	—	—

lane 0 最终得到 $m = 3$，$d = 2e^{-1} + 2$，和直接对 $\{1, 3, 2, 0\}$ 求 e^{x-3} 之和一致。

6. V3 寄存器缓存方案的适用条件与寄存器压力限制

方案：既然第二次遍历只是为了「再读一遍 x 再算 e^{x-m}」，那把 x 留在寄存器里不就行了。V3/V5 用 `float4 reg_cache[4]` 缓存每线程负责的 16 个元素（$4096/256 = 16$），第一遍算 (m, d) 的同时写入 cache，第二遍直接从寄存器算 y。DRAM 流量从 2R+1W 降到 1R+1W。

适用条件：

1. **每线程元素数 $N/\text{blockDim} \le \text{MAX_ELEMS_PER_THREAD} = 16$**。要缓存整行，就必须把整行装进寄存器；装不下就只能回落到 V2。
2. **循环必须能完全展开、下标必须是编译期常量**。这一点 V3 的文件头记录了一个非常典型的坑：
   - 写法 A：`int i = tid + k*blockDim.x;` → `reg_cache[k]` 可静态展开 → **0.260 ms / 322 GFLOPS**
   - 写法 B：用运行期的值（如 `i / blockDim.x`）算下标 → 编译器无法把数组提升为寄存器，只能落到 **local memory**（本质是 global memory 的一段），缓存直接失效 → **0.502 ms / 167 GFLOPS**

   同一份算法差了一倍性能，差别只在下标是不是编译期常量能确定。`#pragma unroll` + 额外单独维护 `i` 就是为了保证这一点。
3. **N 能被 4 整除（`float4` 对齐）**。代码里 `N4 = N/4` 后面还留了标量尾循环处理余数。

寄存器压力（本机 `-Xptxas -v` 实测，sm_75）：

版本	方案	寄存器/线程	理论占用率（block=256，1024 线程/SM）
V1	block 归约	26	100%
V2	V1 + float4	34	100%
V3	V2 + 寄存器缓存	58	100%（踩线）
V5	V3 + grid-stride	63	100%（踩线）

关键数字：Turing 每个 SM 有 64K 个 32-bit 寄存器、最多驻留 1024 线程，所以**满占用率预算恰好是 64 寄存器/线程**。V3 的 58、V5 的 63 都紧贴这条线——文件里把 `MAX_ELEMS_PER_THREAD` 定成 16 不是随便取的，再大 1 就会越线，占用率从 100% 掉到 75%（80 寄存器 → 65536/80 = 819 → 向上取整到 768 线程 = 3 个 block）。

取舍：缓存换掉一次 global load，但吃掉寄存器 → 可能降低 occupancy → 降低延迟隐藏能力。所以它是一个「N 不大、寄存器有富余」时才划算的方案：N 增大时，缓存元素线性增长而占用率断崖下降，收益会被抵消。**每线程元素数是这个方案的核心参数**，也是第 8 条选型的主要依据。

7. Online Softmax 与 FlashAttention 分块策略的数学联系

FlashAttention 的 forward 里，attention 输出是

$$O = \text{softmax}(QK^\top)V = \frac{\sum_j e^{s_j - m} v_j}{\sum_j e^{s_j - m}}, \qquad s_j = q \cdot k_j$$

其中 $m = \max_j s_j$。**注意这不是对一维向量求 softmax，而是「以 e^{s_j-m} 为权重对 v_j 做加权平均」**——这恰恰是 Online Softmax 的泛化：求和对象从标量 1 变成了向量 v_j。

分块策略：FlashAttention 不把整行 K/V 放进 SRAM，而是按列切成 $K^{(1)}, V^{(1)}, \dots, K^{(T)}, V^{(T)}$，逐块计算并累加到 O。设处理完第 t 块后的运行状态为 (m_t, ℓ_t, O_t)，收到第 $t+1$ 块时：

$$m_{t+1} = \max(m_t,\ m^{(t+1)}), \qquad \alpha = e^{m_t - m_{t+1}}$$

$$\ell_{t+1} = \ell_t \cdot \alpha + \ell^{(t+1)}, \qquad O_{t+1} = O_t \cdot \alpha + O^{(t+1)}$$

α 就是第 2 条递推里的 $e^{m_k - m_{k+1}}$，ℓ 就是 d，只是多了一个同样要被 α 缩放的向量累加器 O。 也就是说：

- FlashAttention = 分块 + Online Softmax 的 (m, d) 递推 + 块内 PV 矩阵乘；
- 归一化（除以 ℓ_T）只在最后做一次，中间块之间只需要传播 (m, ℓ, O) 三个运行量；
- 每个块的 $(m^{(t)}, \ell^{(t)}, O^{(t)})$ 可以独立计算，块间合并用的正是第 4 条那个**结合算子**——这就是它能做到「$O(N)$ 显存、不物化 $N \times N$ 注意力矩阵」的数学根基。

所以「Online Softmax 与 FlashAttention 分块策略的数学联系」可以一句话回答：**FlashAttention 的 t 块间递推，就是 Online Softmax 从标量 (m, d) 推广到「(m, ℓ, O) 三元组」的版本，本质是同一个结合算子在分块归约中的应用。** 训练里的反向传播沿用了同一套运行统计（LSE 就是 $\log \ell_T + m_T$），所以推导反向时也不需要重算整个注意力矩阵。

8. 根据 M、N 的大小选择 Softmax 实现

数据形状	推荐	理由
$N \le 16 \times \text{blockDim}$，寄存器有余（如 $N \le 4096$、block=256）	**V3 / V5**（寄存器缓存 One-Pass）	整行装得下，1R+1W，最省带宽
N 大（装不进寄存器，如 $N \ge 16384$）	**V2**（两遍 + `float4`）	寄存器放不下就只剩 2R+1W；用向量化把访存指令数打下来
M 极大（行数远超 SM 数，如 $M \gg 10^5$）	**V4/V5**（grid-stride loop）	限制 grid 规模，避免启动开销和尾部不均衡
M 很小、N 巨大（少行长向量）	一行拆多 block + 跨 block 归约（两段式/atomics）	单行填不满 GPU，必须把一行切开并行
$N < \text{blockDim}$（短行）	**V0 式「一线程一行」**	block 内协作反而浪费线程，一线程一行才能让每个线程都干活

判定主干只有两条：

1. **寄存器能不能装下整行**（$N/\text{blockDim} \le \text{MAX_ELEMS_PER_THREAD}$）→ 决定 1R+1W 还是 2R+1W；
2. **并行度够不够**（M 是否远小于「SM 数 × 每 SM 可驻留 block 数」）→ 决定要不要拆行 / grid-stride。

$M = N = 4096$ 属于「寄存器装得下 + 并行度充足」，所以 V3 最快。

9. 用 Nsight Compute 对比 Two-Pass 与 Three-Pass 的内存吞吐

先看理论上该差多少。 就 $M = N = 4096$（$M \cdot N \cdot 4\text{B} = 67.1$ MB）而言：

方案	读	写	DRAM 总流量	相对三遍
Three-Pass	3 × 67.1 = 201.3 MB	67.1 MB	**268.4 MB**	1.00x
Two-Pass（V0~V2/V4）	2 × 67.1 = 134.2 MB	67.1 MB	**201.3 MB**	**0.75x**
One-Pass（V3/V5）	1 × 67.1 = 67.1 MB	67.1 MB	**134.2 MB**	**0.50x**

`dram__bytes_read.sum` 应当如实反映这个 3 : 2 : 1 的比例，这是「验证优化是否真的生效」最直接的指标。**不要用 wall time 判断——时序受 occupancy 干扰极大。**

实测命令（`ThreePass.cu` 已放在本目录）：

```
nvcc -O3 -arch=native ThreePass.cu -o ThreePass         # 三遍基线
ncu --launch-skip 3 --launch-count 1 \
    --metrics dram__bytes_read.sum,dram__bytes_write.sum,\
              gpu__time_duration.sum,\
              dram__throughput.avg.pct_of_peak_sustained_elapsed \
    ./ThreePass
# 再对两遍版跑同一条命令
ncu --launch-skip 3 --launch-count 1 \
    --metrics dram__bytes_read.sum,dram__bytes_write.sum,\
              gpu__time_duration.sum,\
              dram__throughput.avg.pct_of_peak_sustained_elapsed \
    ./V2
```

指标怎么读：

- `dram__bytes_read.sum` / `dram__bytes_write.sum`：真实 DRAM 存取字节数，直接和三遍基线对比，验证 0.75x / 0.5x 的比例；
- `dram__throughput.avg.pct_of_peak_sustained_elapsed`：DRAM 带宽利用率。若它已经接近 90%+，说明是带宽瓶颈，继续优化指令没有意义、只能减字节；若只有 50%，说明瓶颈在延迟/occupancy，这时该调 block 大小或加并行度；
- `gpu__time_duration.sum`：kernel 实际耗时，用来交叉验证；
- 想进一步定位，可以加 `l1tex__t_bytes_pipe_lsu_mem_global_op_ld.sum`（L1→SM 的 load 流量）与 `sm__warps_active.avg.pct_of_peak_sustained_active`（占用率）。当 `dram__bytes` 明显小于「load 流量 × 元素大小」时，说明命中 L2 了。

> 本机说明：这台机器上的 `ncu` 只是 conda 里的一个壳，真正的 Nsight Compute 目录 `/opt/nvidia/nsight-compute` 不存在，所以尚未跑出实测 profile，上面的数字是解析计算值。装好 Nsight Compute 后按上述命令即可复现。

一个必须注意的公平性陷阱：`ThreePass.cu` 沿用了 V0 的结构（一线程一行），只有 16 个 block、4096 个线程，严重欠占用。我实测三遍版 3.863 ms，而 V0（两遍、同样结构）3.287 ms——**这个 15% 的差距远小于 25% 的流量差距**，因为两者都受限于延迟而非带宽（有效带宽只有 ~60–70 GB/s，不到峰值 616 GB/s 的 12%）。

正确做法是**让两个版本只有「扫描遍数」一个变量不同**：三遍版也应改写成「一个 block 一行 + warp 归约」再测。否则 ncu 里 `dram__throughput` 的差异会主要反映占用率，而不是扫描遍数。更严格地说：只要 `dram__throughput.pct_of_peak < ~70%`，这个 kernel 就是延迟受限而不是带宽受限，此时「少读一遍」的收益会被访存延迟掩盖。

10. 把 (m, d) 递推思路扩展到其他增量计算场景

这个递推的通用形式是：**「先累加、事后用最终基准统一缩放」**。凡是「逐项累加的结果需要以一个依赖全体数据的基准做归一化/缩放」的统计量，都能套用。核心结构：

```
运行状态 (base, acc)；每来一个新项 x：
    base_new = combine(base_old, base_of(x))
    acc_new  = acc_old * rescale(base_old, base_new) + acc_of(x) * rescale(base_of(x), base_new)
```

可以直接迁移的场景：

1. **Streaming / chunked 归一化**：任何「分块计算 + 事后归一化」的算子。典型如 online layer norm、streaming attention（KV cache 场景下，每来一个新的 KV block 就要更新一次 m, ℓ, O，正是 FlashDecoding 的做法）。
2. **Welford 在线均值/方差**：这是 (m, d) 递推在二阶矩上的直接对应物。Welford 的 $\bar{x}_k = \bar{x}_{k-1} + \frac{x_k - \bar{x}_{k-1}}{k}$ 之所以数值稳定，和 Online Softmax 之所以稳定是同一个原因——**增量修正而非「先存全部再统一算」**。
3. **Log-Sum-Exp / LogSpace 累加**：把状态存成 $(m, \log d)$ 就能把乘法变成加法，数值上更稳。CRF 的前向算法、HMM 的 forward-backward、语言模型的 log-prob 累积、beam search 的分数合并，用的都是 LSE 版本的 (m, d) 递推。把 d 换成 $\log d$ 后合并式变成 $\mathrm{LSE}(a, b) = m^* + \log(e^{a-m^*} + e^{b-m^*})$，和这里完全同构。
4. **混合模型 / 高斯加权和**：把「和」换成加权求和、e^{x_i} 换成各分量的权重，就得到 EM 里分量的在线更新。
5. **任何可结合 + 需要基准对齐的归约**：只要某个状态量满足「先合并后缩放 ≡ 先各自缩放后合并」，它就能写成这种二元组，从而支持树形/分块/多级并行。常见的还有 max+argmax、min+argmin、加权平均 + 权重和。

判别标准很简单：**如果把「基准」定错，后面的项需要被整体缩放，那么这个量就适合 Online 递推。** 反例是「结果的每一项都独立、不需要统一基准」的算子（比如单纯的求和、元素的逐点乘），它们本来就没有这个问题，也就不需要这套结构。

三、本目录版本演进与实测

本机重跑（`M=N=4096`，block=256，200 次取平均）：

版本	核心改动	时间 (ms)	GFLOPS	有效带宽
V0	一线程一行，Online 两遍（grid=16）	3.287	25.5	~61 GB/s
V1	一 block 一行 + shuffle/shared 两级归约	0.373	225.2	~540 GB/s
V2	V1 + `float4` 向量化	0.358	234.2	~563 GB/s
V3	V2 + 寄存器缓存 One-Pass（展开为编译期常量）	**0.260**	**322.1**	~515 GB/s
V3（下标为运行期值）	同上，但 `reg_cache` 退化成 local memory	0.502	167.0	—
V4	V2 + grid-stride loop	0.354	234.2	~569 GB/s
V5	V3 + grid-stride loop	0.282	—	~476 GB/s
ThreePass	三遍 Safe Softmax 基线（同 V0 结构）	3.863	21.7	~69 GB/s

怎么读这张表：

- **V0 → V1 是数量级的跨越**（8.8x），但收益不来自「少读一遍」，而来自**占用率**：grid 从 16 个 block 变成 4096 个 block，GPU 终于被填满。这说明对 memory-bound 算子，先解决并行度、再谈减少字节。
- **V1 → V2 只快 4%**，因为两者都已经在 DRAM 峰值附近（V2 有效带宽约 563 GB/s，达到 2080 Ti 峰值 ~616 GB/s 的 91%）。**已经贴着 roofline 的算子，向量化只能省指令，不能省字节，收益自然有限。**
- **V2 → V3 再快 27%**，因为这次真正减少了访存字节（201.3 → 134.2 MB，理论 33%）。这是本组里唯一一次「算法层面」的优化。
- **V5 略慢于 V3**（0.282 vs 0.260）：两者算法相同，差别在 V5 加了外层 grid-stride 与每行末尾的 `__syncthreads()`，且寄存器 63 vs 58。这提醒：grid-stride 的价值在大规模、Block 数受限的场景，$M = 4096$ 时它反而带来额外同步开销。
- **AR 一律是 0.625 FLOPs/Byte**：因为各版本 `main()` 里都按理想的 2×M×N×4 字节算。它只反映问题的算术强度，**不能反映真实流量**——真实流量要看第 9 条的 `dram__bytes_*.sum`。

四、两个已知问题

1. **V3.cu / V5.cu 的 grid-stride 上界写成了 `N`，应为 `M`**：

   ```
   for (int row = blockIdx.x; row < N; row += gridDim.x)   // V3/V5: 应为 M
   for (int row = blockIdx.x; row < M; row += gridDim.x)   // V4: 正确
   ```

   当前测试恰好 $M = N = 4096$ 所以掩盖了问题。当 $M < N$ 时会把 x 读出行边界（越界），$M > N$ 时会漏算后半部分的行；$M > N$ 且 N 不是 `gridDim.x` 的整数倍时还会写出越界。

2. **`float4* x4 = reinterpret_cast<float4*>(x)` 假设了行首 16 字节对齐**。行跨度为 $N \cdot 4$ 字节，$N = 4096$ 时恰好是 64 的倍数，没问题；但 N 不是 4 的倍数时，第 k 行的行首就不满足 `float4` 对齐要求，会被静默地读错。尾循环只处理了元素个数余数，没有处理对齐余数——工程实现里应改成标量路径或补齐 stride。